

Review



- Last Class:
 - Register renaming to resolve false dependences.
- Today's class:
 - Branch prediction

Control dependences

- Determine the order of instructions with respect to branches.

**if P1 then S1 ;
if P2 then S2 ;**

S1 is control dependent on P1 and
S2 is control dependent on P2 (and P1 ??).

- An instruction that is control dependent on P cannot be moved to a place where it is no longer control dependent on P , and visa-versa

Example :

```
add    R1,R2,R3
beq     R4, R0, L
sub     R1,R1,R6
L:  ...
or      R7,R1,R8
```

“or” instruction depends
on the execution flow

Branch prediction (§3.3)



- Static branch prediction (built into the architecture)
 - The default is to assume that branches are not taken
 - May have a design which predicts that branches are taken
- It is reasonable to assume that
 - forward branches are often not taken
 - backward branches are often taken
- May come up with more static predictors based on branch directions.
- Profiling is the standard technique for predicting the probability of branching.
- Dynamic predictors rely on the history to predict the future branch direction

Static Branch Prediction

- ❑ Resolve branch hazards by **assuming** a given outcome and proceeding without waiting to see the actual branch outcome
- 1. **Predict not taken** – *always* predict branches will **not** be taken, continue to fetch from the sequential instruction stream, only when branch *is* taken does the pipeline stall
 - ❑ If taken, **flush** instructions **after** the branch (earlier in the pipeline, later in the code)
 - in IF, ID, and EX stages if branch logic in MEM – **three** stalls
 - In IF and ID stages if branch logic in EX – **two** stalls
 - in IF stage if branch logic in ID – **one** stall
 - ❑ ensure that those flushed instructions haven't changed the machine state – automatic in the MIPS pipeline since machine state changing operations are at the tail end of the pipeline (MemWrite (in MEM) or RegWrite (in WB))
 - ❑ restart the pipeline at the branch destination

Static Branching Structures

- ❑ Always predict NT works well for “top of the loop” branching structures

- ❑ But such loops have jumps at the bottom of the loop to return to the top of the loop – and incur the jump stall overhead every time through the loop

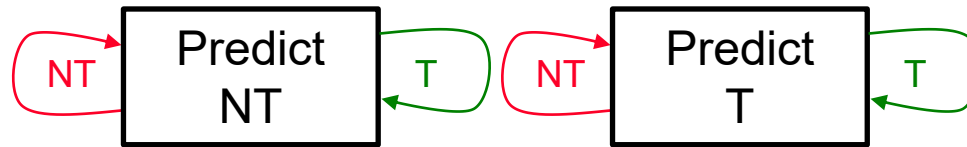
```
Loop: beq $1,$2,Out
      1nd loop instr
      .
      .
      .
      last loop instr
      j Loop
Out:  fall out instr
```

- ❑ Always predict NT doesn't work well for “bottom of the loop” branching structures

- ❑ Guess wrong every time through the loop except the last time (when we fall out of the loop)

```
Loop: 1st loop instr
      2nd loop instr
      .
      .
      .
      last loop instr
      bne $1,$2,Loop
      fall out instr
```

0-Bit Predictors, Loop for 10 Iterations



iteration	predict	actual	predict	actual
1	NT	T	T	T
2	NT	T	T	T
3	NT	T	T	T
4	NT	T	T	T
5	NT	T	T	T
6	NT	T	T	T
7	NT	T	T	T
8	NT	T	T	T
9	NT	T	T	T
10	NT	NT	T	NT
	10% accuracy		90% accuracy	

```

pre-loop instr
Loop:
  1st loop instr
  2nd loop instr
    .
    .
    .
  last loop instr
  bne $1,$2,Loop
post-loop instr
  
```

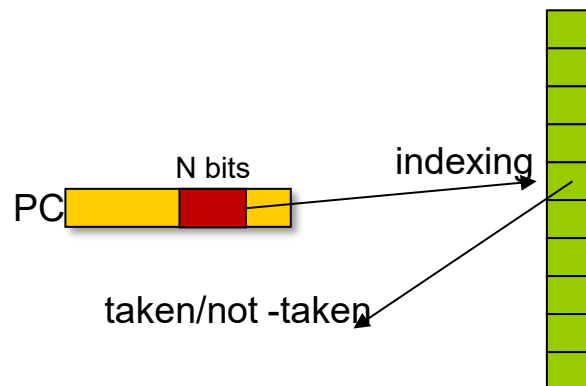
Other Branch Prediction Possibilities

2. **Predict taken (T)** – predict branches will always be taken
 - ❑ BUT predict taken *always* incurs one stall cycle (*if* branch destination hardware has been moved to the ID stage)
 - ❑ Is there a way to “cache” the **address** of the branch target instruction, or *better yet* the **actual** branch target **instruction** itself ?? Yes! **Branch Target Buffer (BTB)**
- ❑ As the branch penalty increases (for deeper pipelines), a simple static prediction scheme will hurt performance. With more hardware, it is possible to try to predict branch behavior **dynamically** during program execution
3. **Dynamic branch prediction** – predict branches at run-time using *run-time* information

Dynamic Branch Prediction

- Different than attaching a prediction with each branch
- Performance depends on the accuracy of prediction and the cost of miss-prediction.
- **Branch prediction buffer (Branch history table - BHT):**
 - 1-bit table (cache) indexed by some bits of the address of the branch instructions (can be accessed in decode stage) → hashing
 - Record whether or not the branch was taken last time
 - May have collision (how to resolve? Cuckoo hashing, overheads?).
 - Will cause two miss-predictions in a loop (at start and end of loop).

L1:
.....
.....
beqz R1, L1



Dynamic Branch Prediction Buffer

- ❑ A **branch prediction buffer** (aka branch history table (**BHT**)) in the IF stage addressed by the low order bits of the PC, contains bit(s) (passed to the ID stage through the IF/ID pipeline register) that tell whether the branch was taken or not the last time it was executed
- ❑ 4,096 entry table programs vary from 1% misprediction (nasa7, tomcatv) to 18% (eqntott), with spice at 9% and gcc at 12%
- ❑ 4,096 about as good as infinite table, but 4,096 is a lot of hardware

Branch History Table

- ❑ The BHTs prediction bits may predict incorrectly (may be a wrong prediction for this branch this iteration or may be from a different branch with the same low order PC bits) but that doesn't affect **correctness**, just **performance**
 - ❑ Branch decision occurs in the ID stage after determining that the fetched instruction is a branch and checking the prediction bit(s)
 - ❑ Problems? (side-channel security!)
 - ❑ Evtvushkin, Dmitry, Ryan Riley, Nael CSE and ECE Abu-Ghazaleh, and Dmitry Ponomarev. "**Branchscope: A new side-channel attack on directional branch predictor.**" ACM SIGPLAN Notices 53, no. 2 (2018): 693-707.
 - ❑ Abu-Ghazaleh, Nael, Dmitry Ponomarev, and Dmitry Evtvushkin. "**How the spectre and meltdown hacks really worked.**" IEEE Spectrum 56, no. 3 (2019): 42-49.
- ❑ If the prediction is wrong, flush the incorrect instruction(s) in pipeline, restart the pipeline with the right instruction, and invert the prediction bit(s)

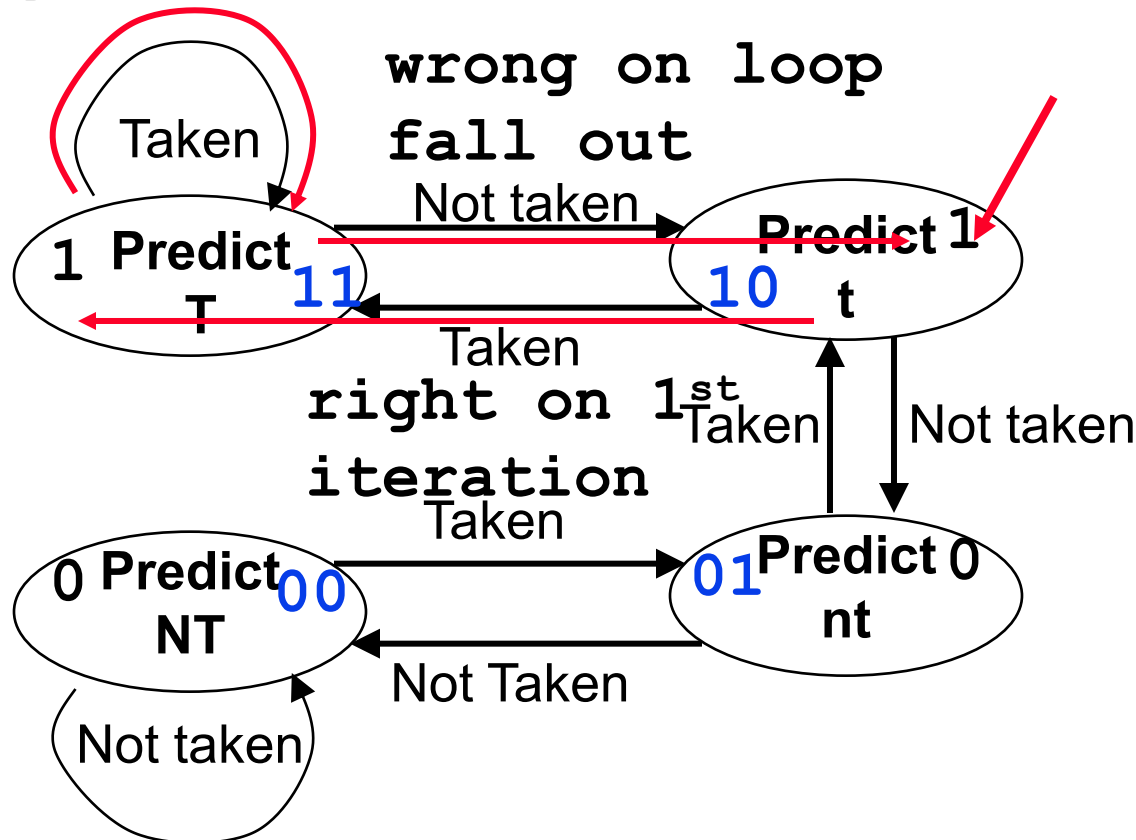
But Also Must Know the Branch Target

- ❑ A **branch target buffer (BTB)** in the IF stage can cache the branch target **address**, but remember we also need to fetch the next sequential instruction (at PC+4).
 - ❑ Would need a two read port IM
- ❑ Or, the BTB can cache the actual branch target **instruction** while the Instruction Memory is fetching the next sequential instruction
 - ID stage can then select between PC+4 and branch target instruction
- ❑ If we predict correctly, stalls can be avoided no matter which direction the branch goes

2-bit Dynamic Branch Predictors

- A 2-bit scheme can give 90% accuracy since a prediction must be wrong twice before the prediction bit is changed

right 9 times



```

Loop: 1st loop instr
      2nd loop instr
      .
      .
      .
      last loop instr
      bne $1,$2,Loop
      fall out instr
  
```

2-bits branch predictors

- change your prediction only if you miss-predict twice
- helps if a branch changes directions occasionally (ex. Nested loops)
- In general, n -bit predictors are called *Local Predictors*.
 - Use a saturated counter (++ on correct prediction, -- on wrong prediction)
 - n -bit prediction is not much better than 2-bit prediction ($n > 2$).
 - A BHT with 4K entries is as good as an infinite size BHT

Correlating Branch predictors (global predictors)

- Hypothesis: recent branches are correlated (behavior of recently executed branches affects prediction of current branch).

- Example 1:

<pre> if (a == 2) a = 0 ; if (b == 2) b = 0; if (a != b) ... </pre>		<pre> addi R3,R1, -2 bnez R3, L1 ... add R1, R0, R0 L1: addi R3, R2, -2 bnez R3, L2 add R2, R0, R0 L2: sub R3, R1, R2 beqz R3, L3 </pre>	 B1  B2  B3
---	---	---	--

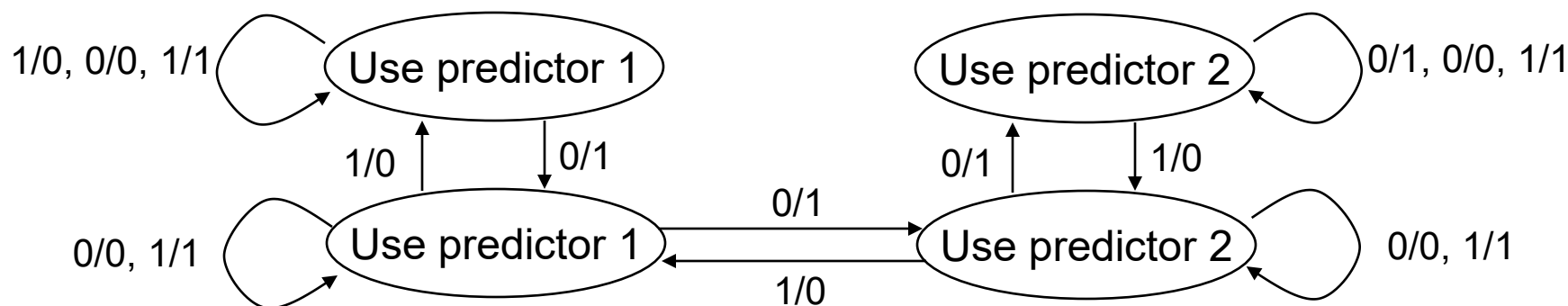
If B1 is not taken and B2 is not taken, then B3 will be taken

If B1 and B2 are taken, then B3 will probably not be taken,

- Example 2: `if (d == 0) d = 1 ;`
 `if (d == 1) .....`

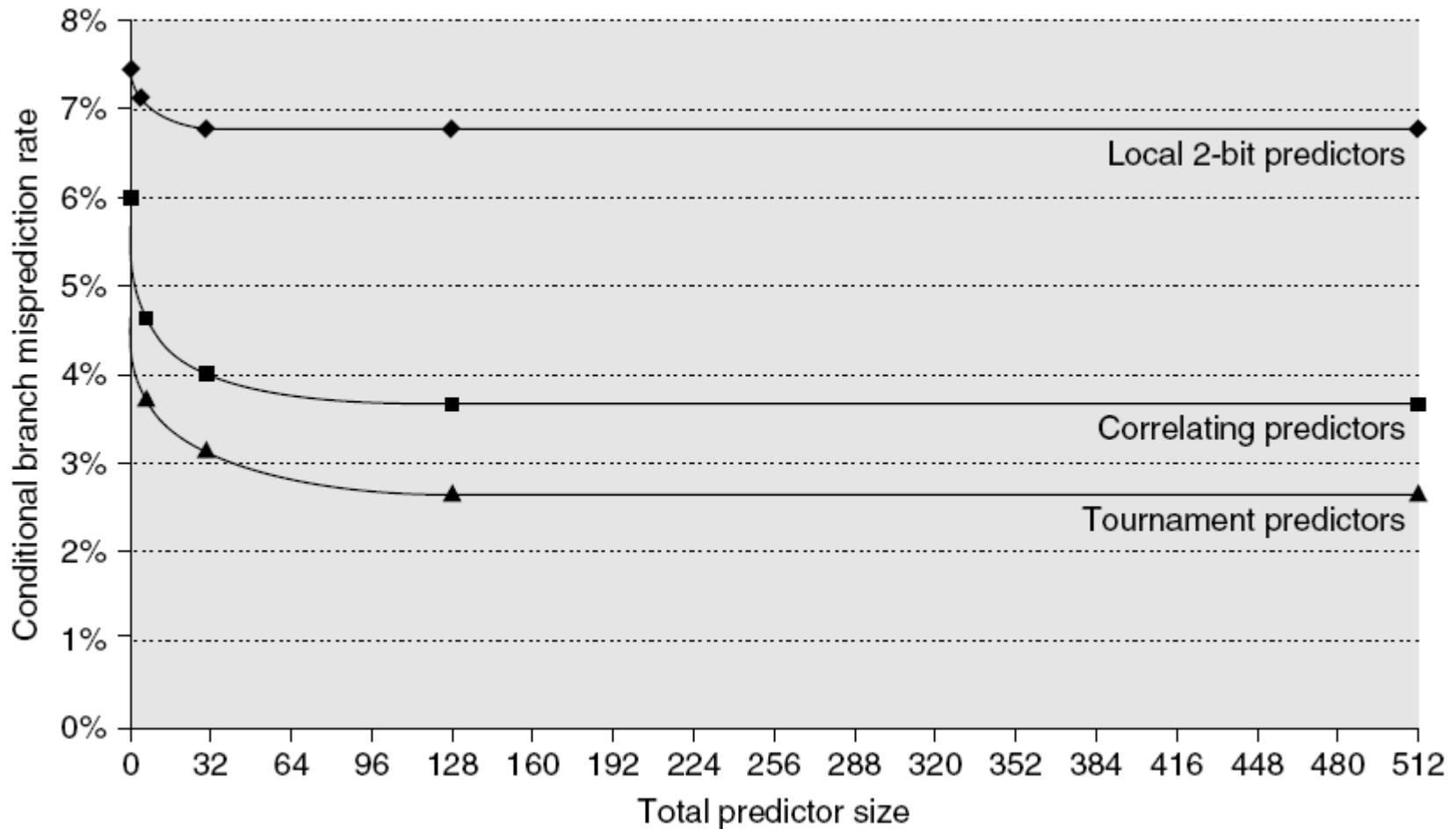
Tournament predictor (hybrid local-global predictors)

- Combines a global predictor and a local predictor with a strategy for selecting the appropriate predictor (multilevel predictors).



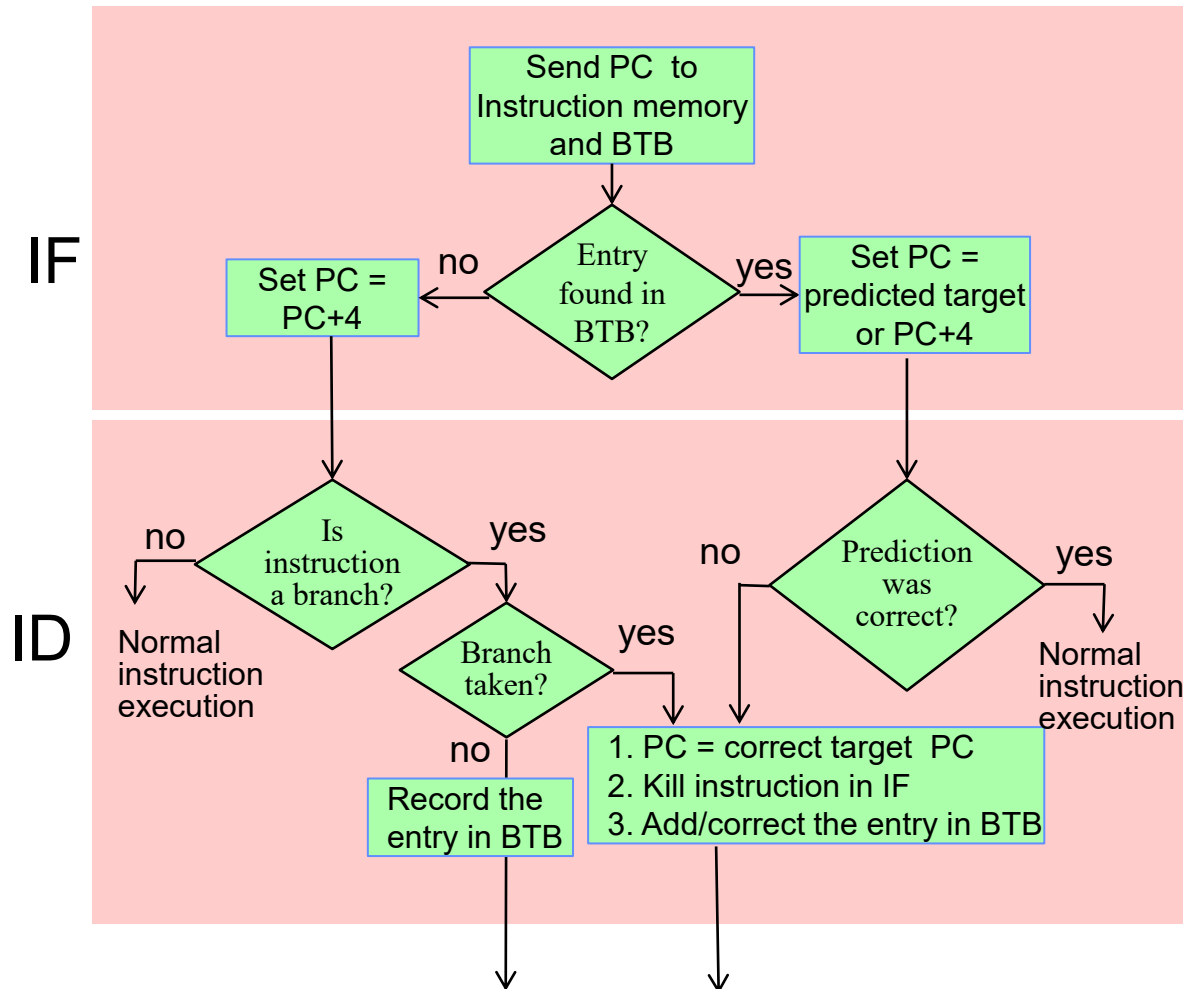
p_1/p_2 == predictor 1 is correct/predictor 2 is correct

Performance of Branch predictors



Branch prediction & pipelining

Assuming that branch condition and target are resolved in ID stage



A similar chart may be drawn if branch condition/target are resolved in EX

Branch prediction (§3.3)



- Machine learning in branch prediction?
 - Calder, Brad, Dirk Grunwald, Michael Jones, Donald Lindsay, James Martin, Michael Mozer, and Benjamin Zorn. "Evidence-based static branch prediction using machine learning." ACM Transactions on Programming Languages and Systems (TOPLAS) 19, no. 1 (1997): 188-222.
 - Jiménez, Daniel A., and Calvin Lin. "Dynamic branch prediction with perceptrons." In Proceedings HPCA Seventh International Symposium on High-Performance Computer Architecture, pp. 197-206. IEEE, 2001.
 - Jiménez, Daniel A., and Calvin Lin. "Neural methods for dynamic branch prediction." ACM Transactions on Computer Systems (TOCS) 20, no. 4 (2002): 369-397.

Review: Pipeline Hazards

❑ Structural hazards

- ❑ Add more hardware resources
- ❑ Design pipeline to eliminate structural hazards

❑ Data hazards – read after write (RAW)

- ❑ Use data forwarding inside the pipeline
- ❑ For those cases that forwarding won't solve (e.g., load-use) include hazard hardware to insert stalls in the instruction stream

❑ Control hazards – `beq, bne, j, jr, jal, jalr`

- ❑ Move decision point as early in the pipeline as possible (e.g., Decode) – reduces number of stalls at the cost of additional hardware
- ❑ Predict – with dynamic branch prediction, can reduce the impact of control hazard flushes even further if the branch prediction (contained in a BHT in the Fetch stage) is correct (shoot for 97%+ accuracy) and if the branched-to instruction is cached (in a BTB also in the Fetch stage)

Roadmap

- ❑ Now: static parallelism
- ❑ Next: dynamic parallelism

Why OOO?

- ❑ Consider the following instruction sequence:

I1

I2

I3

- ❑ If we do not employ OOO execution and I2 is stalled (if, for example, it depends on I1), I3 will be stalled as well
- ❑ An instruction is stalled because of an *irrelevant instruction*
 - ❑ A consequence of in-order execution
- ❑ **Solution:** Let I3 get scheduled and execute while I2 is waiting – out-of-order execution
 - ❑ Improves utilization and performance

Why OOO?

- ❑ What do the following two pieces of code have in common (with respect to execution in the previous design)?

```
IMUL R3 ← R1, R2
ADD  R3 ← R3, R1
ADD  R9 ← R6, R7
IMUL R5 ← R6, R8
ADD  R7 ← R3, R5
```

```
LD   R3 ← R1 (0)
ADD  R3 ← R3, R1
ADD  R9 ← R6, R7
IMUL R5 ← R6, R8
ADD  R7 ← R3, R5
```

- ❑ Answer: First ADD stalls the whole pipeline!

- ❑ ADD cannot dispatch because its source registers unavailable
- ❑ Later **independent** instructions cannot get executed

- ❑ How are the above code portions different?

- ❑ Answer: Load latency is variable (unknown until runtime)
- ❑ What does this affect? Think compiler vs. microarchitecture
- ❑ Can compiler resolve the issue?

Out-of-order Execution (Dynamic Scheduling)

- ❑ Idea: Move the dependent instructions out of the way of independent ones
 - ❑ Rest areas for dependent instructions: Reservation Stations
- ❑ Monitor the source “values” of each instruction in the resting area
- ❑ When all source “values” of an instruction are available, “fire” (i.e. issue) the instruction
 - ❑ Instructions dispatched in dataflow (not control-flow) order
- ❑ Benefit:
 - ❑ Latency tolerance: Allows independent instructions to execute and complete in the presence of a long latency operation

Dynamic OOO Datapaths

- ❑ **Scoreboarding** – CDC 6600 (Thornton) first publication in 1964
 - ❑ Named after CDC 6600 scoreboard
 - ❑ Used **centralized** hazard detection logic (**scoreboard**) to support OOO execution. Instr's were stalled when their FU was busy, for RAW dependencies, **and for WAW and WAR dependencies**
- ❑ **Tomasulo** – IBM 360/91 (Tomasulo) first publication in 1967
 - ❑ Used **distributed** hazard detection logic (**reservation stations** feeding each FU) to support OOO execution with **register renaming** that eliminated WAW and WAR dependencies; distributed results from FUs to reservation stations on a Common Data Bus
 - ❑ Writes results to register file and memory when instr's completes – possibly out-of-order – so **could not support speculative execution** (e.g., branch speculation)

Using Scoreboards (see Appendix C.7):

- Dates to the first supercomputer, the CDC 6600 in 1963
- Split the ID stage into
 - Issue - decode and check for structural hazards,
 - Read operands → wait until no data hazards, then read operands.
- Instructions wait in a queue and may move to the EX stage (dispatched) out of order.

Scoreboard information (3 tables)



- **Instruction status:**
 - issued, read operands and started execution (dispatched), completed execution or wrote result,
- **Functional unit status** (assuming non-pipelined units)
 - busy/not busy
 - operation (if unit can perform more than one operation)
 - destination register - F_i
 - source registers (containing source operands) - F_j and F_k
 - the unit producing the source operands (if stall to avoid RAW hazards) - Q_j and Q_k
 - flags to indicate that source operands are ready -- R_j and R_k
- **Register result status:**
 - indicates the functional unit that contains an instruction which will write into each register (if any)

Four stages of scoreboard control



- **Issue only if no structural, WAR or WAW hazards.**
 - Issue (and reserve the functional unit) if the functional unit is free and
 - » No issued or dispatched instruction (in state “issued” or “dispatched”) will write to the destination register (to avoid WAW)
 - » No issued instruction (in state “issued”) will read from the destination register (to avoid WAR)
 - otherwise, stall, and block subsequent instructions
 - the fetch unit stalls when the queue between the fetch and the issue stages is full (may be only one buffer).
- **Read operands only if no RAW hazard.**
 - If a RAW hazard is detected, wait until the operands are ready,
 - When the operands are ready, read the registers and move to the execution stage,
 - Note that instructions may proceed to the EX stage out-of-order.
- **Execution.**
 - When execution terminates, notify the score board.
- **Write result to register file**

Example:

Instruction
status

Instruction		Issue	Read op.	Exec. Completed	Write result	
fld	F6, 34(R2)	X	X	X	X	done
fld	F2, 45(R3)	X	X	X		
fmul.d	F0, F2, F4	X				
fsub.d	F8, F6, F2	X				
fdiv.d	F10, F0, F12	X				
fadd.d	F6, F8, F2					Not in

Example:

Instruction
status

Instruction		Issue	Read op.	Exec. Completed	Write result	
fld	F6, 34(R2)	X	X	X	X	done
fld	F2, 45(R3)	X	X	X		
fmul.d	F0, F2, F4	X				
fsub,d	F8, F6, F2	X				
fdiv.d	F10, F0, F12	X				
fadd.d	F6, F8, F2					Not in

Func. unit
status

Unit	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Int-m	Yes	Load	F2	R3				Yes	
Mult1	Yes	Mult	F0	F2	F4	Int.		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2		Int.	Yes	No
divide	Yes	Div	F10	F0	F12	Mult1		No	Yes

Example:

Instruction status	Instruction		Issue	Read op.	Exec. Completed	Write result	done
	fld	F6, 34(R2)	X	X	X	X	
	fld	F2, 45(R3)	X	X	X		
	fmul.d	F0, F2, F4	X				
	fsub,d	F8, F6, F2	X				
	fdiv.d	F10, F0, F12	X				
	fadd.d	F6, F8, F2					Not in

Func. unit status	Unit	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
	Int-m	Yes	Load	F2	R3				Yes	
	Mult1	Yes	Mult	F0	F2	F4	Int.		No	Yes
	Mult2	No								
	Add	Yes	Sub	F8	F6	F2		Int.	Yes	No
	divide	Yes	Div	F10	F0	F12	Mult1		No	Yes

Register status	F0		F2	F4	F6	F8	F10	F12	...	F30
	Func. U	Mult1	Int.			Add	Div			

Example: when “fld F2, 45(R3)” is writing

Instruction		Issue	Read op.	Exec. Completed	Write result	done
Instruction status	fld F6, 34(R2)	X	X	X	X	
	fld F2, 45(R3)	X	X	X	X	
	fmul.d F0, F2, F4	X				
	fsub.d F8, F6, F2	X				
	fdiv.d F10, F0, F12	X				
	fadd.d F6, F8, F2					Not in

Unit		Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Func. unit status	Int-m	No								
	Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
	Mult2	No								
	Add	Yes	Sub	F8	F6	F2			Yes	Yes
	divide	Yes	Div	F10	F0	F12	Mult1		No	Yes

Register status		F0	F2	F4	F6	F8	F10	F12	...	F30
Func. U		Mult1				Add	Div			